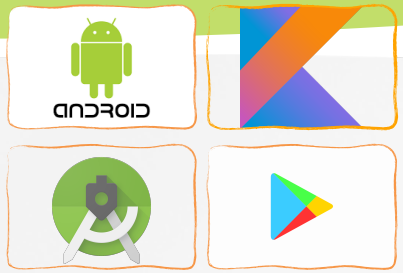
Software Development for Mobile Devices

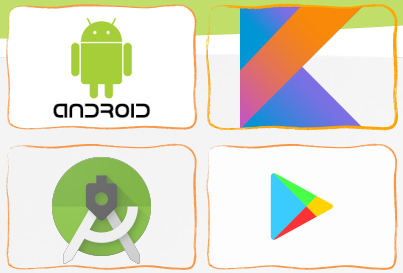
Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

Content



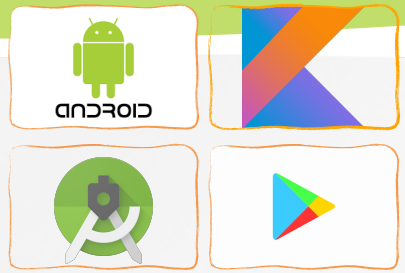
- What is ConstraintLayout
- How to add ConstraintLayout to your project
- Add, remove and edit a constraint
- Parent Position and Order position
- Alignment
- Baseline alignment
- Guideline and Barrier
- Create ConstraintLayout programmatically

ConstraintLayout



- ConstraintLayout allows you to create large and complex layouts with a flat view hierarchy (no nested view groups).
  - Using relationships between sibling views and the parent layout,
  - Drag-and-dropping with Android Studio's Layout Editor instead of editing the XML.
- You can use ConstraintLayout on Android systems starting with API level 9 (>=2.3)
- https://www.youtube.com/watch?v=XamMbnzI5vE

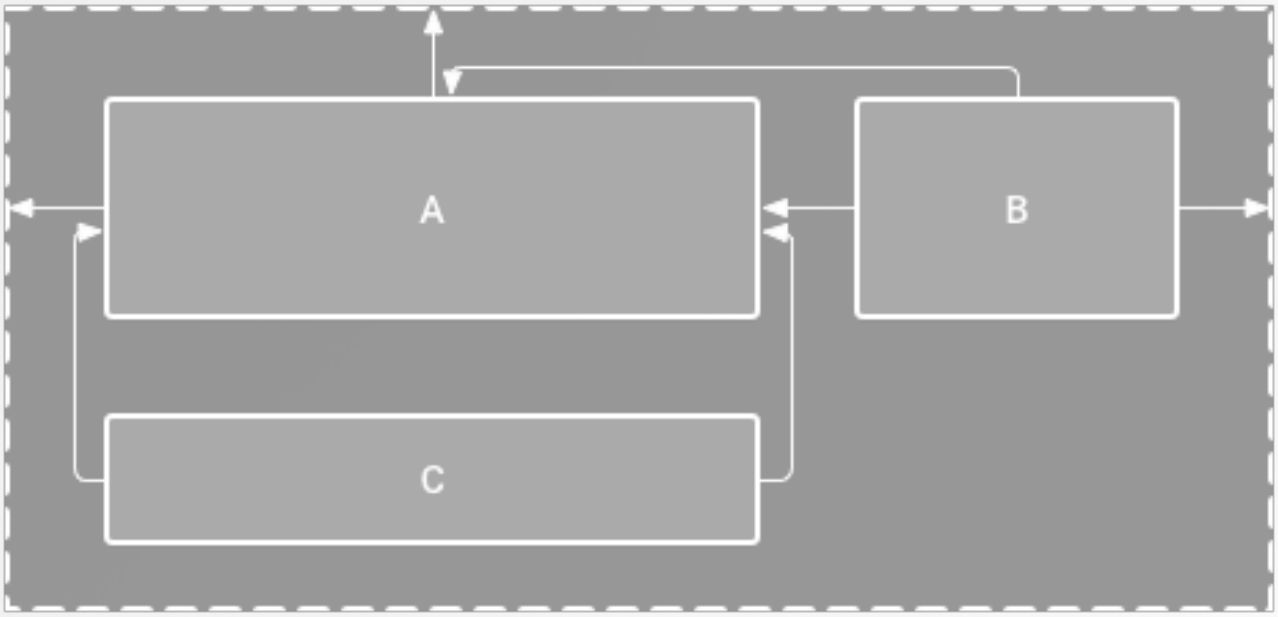
ConstraintLayout



- Defining a view's **position** in ConstraintLayout,
  - **At least 1 horizontal** and **1 vertical constraint** for the view.
  - Each constraint represents a connection or alignment to another view, the parent layout, or an invisible guideline.
- Each constraint defines the view's position along either the vertical or horizontal axis;
  - Each view must have a minimum of 1 constraint for each axis, but often more are necessary.
- If a view has no constraints when you run your layout on a device, it is drawn at position [0,0] (the top-left corner).

ConstraintLayout

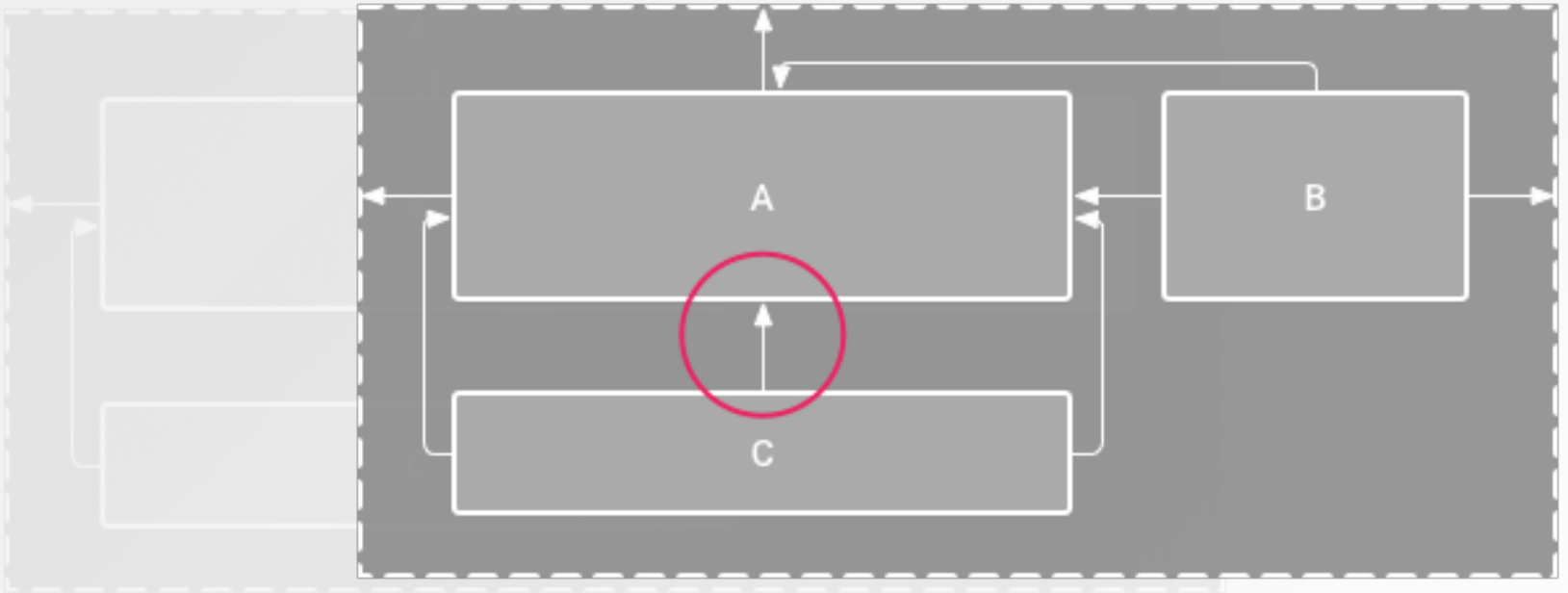
- The layout looks good in the editor, but
  - There's no vertical constraint on view C



- A missing constraint won't cause a compilation error, but we can view the errors and other warnings, click **Show Warnings and Errors** .

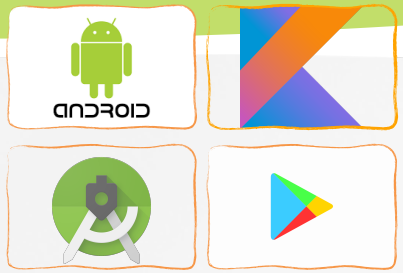
ConstraintLayout

- The layout looks good in the editor, but
  - There's no vertical constraint on view C



- A missing constraint won't cause a compilation error, but we can view the errors and other warnings, click **Show Warnings and Errors** .

Add ConstraintLayout to your project



» Ensure you have the maven.google.com repository declared in your module-level build.gradle file:

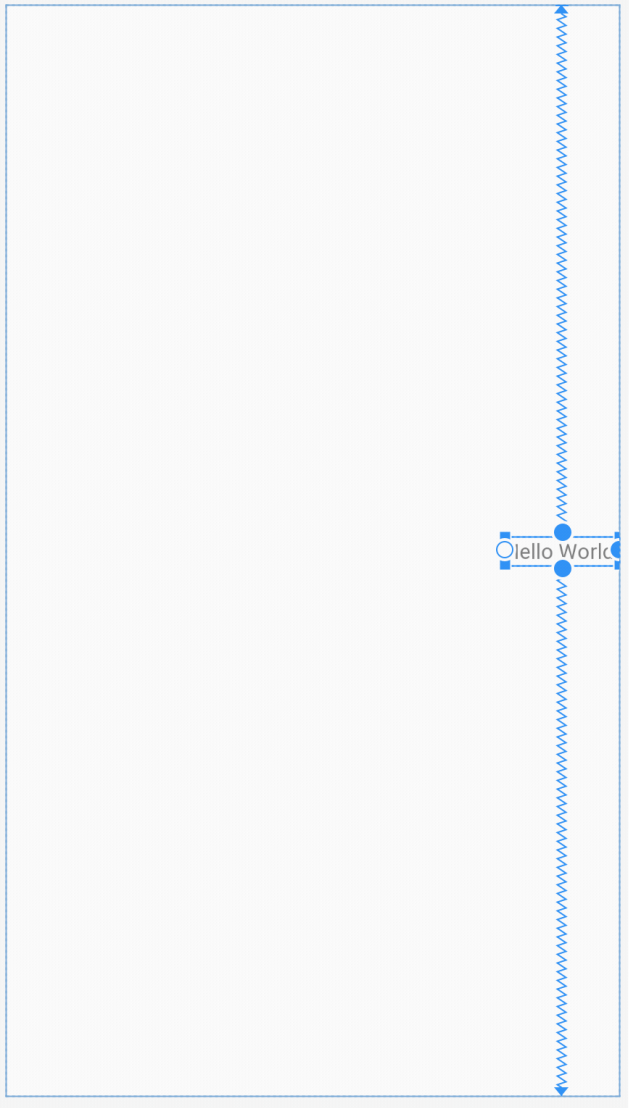
```
repositories {
    google()
}
```

» Add the library as a dependency in the same build.gradle file:

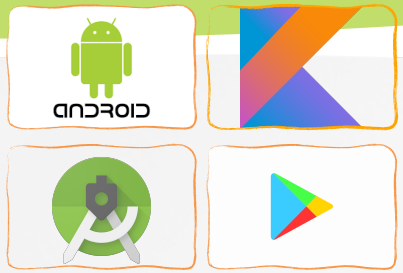
```
dependencies {
    implementation 'androidx.constraintlayout:constraintlayout:1.1.3'
}
```

Add or remove a constraint

- Drag a **view** from the **Palette** window into the editor.
  - A bounding box with square resizing handles on each corner and circular constraint handles on each side.
  - Click a constraint handle and drag it to an available anchor point. (the edge of another view, the edge of the layout, or a guideline…)



Add or remove a constraint

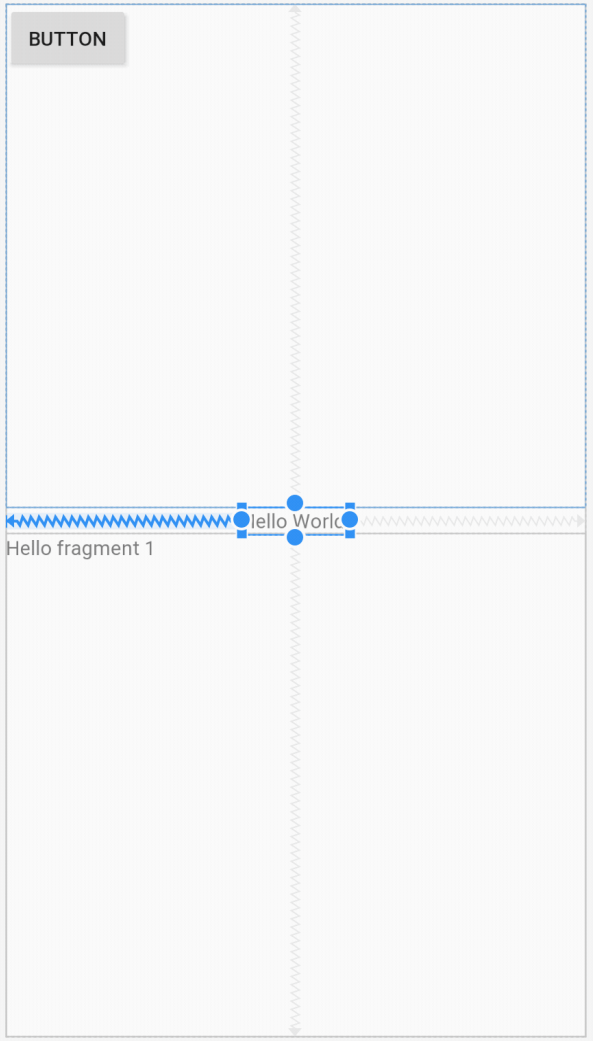


Some rules

- Every view must have at least 2 constraints: 1 horizontal and 1 vertical.
- You can create constraints only between a constraint handle and an anchor point that share the same plane.
  - For example, a vertical plane (the left and right sides) of a view can be constrained only to another vertical plane; and baselines can constrain only to other baselines.
- Each constraint handle can be used for just 1 constraint
- We can create multiple constraints (from different views) to the same anchor point.

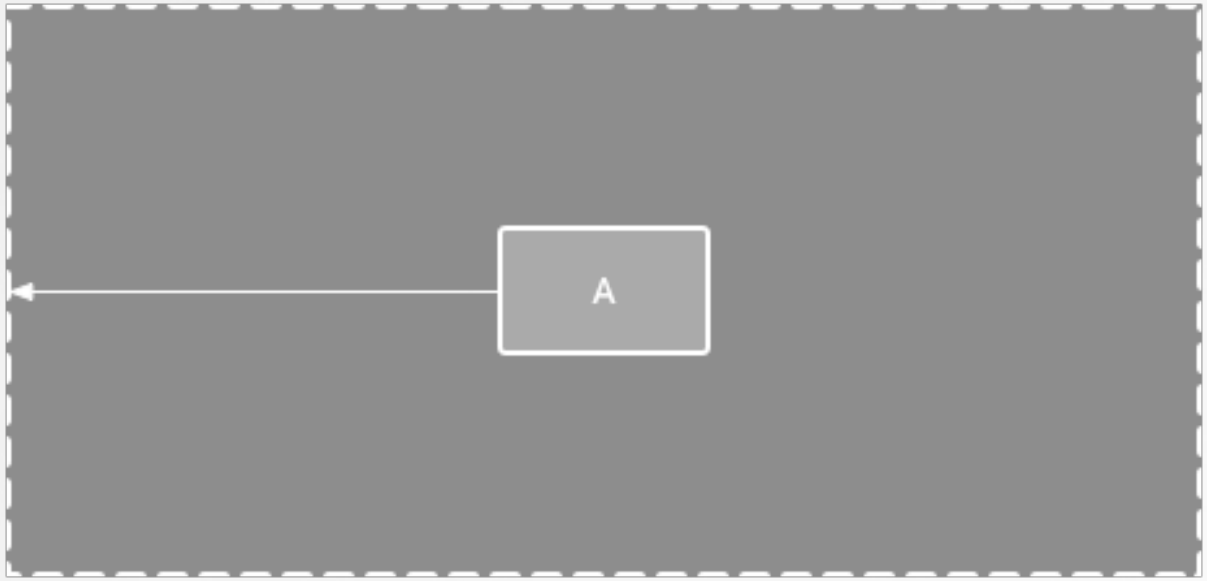
Delete a constraint by

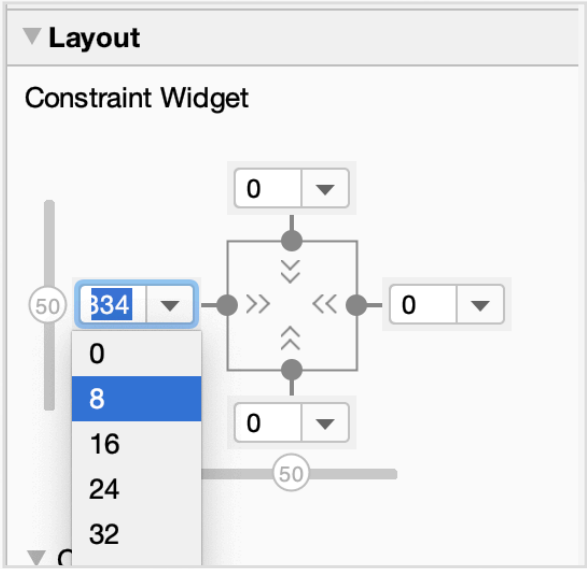
- Click on a constraint to select it, and then press **Delete**.
- **Press** and **hold** **Control** (**Command** on macOS), and then click on a constraint anchor.
  - The constraint turns red to indicate that you can click to delete it



Parent position

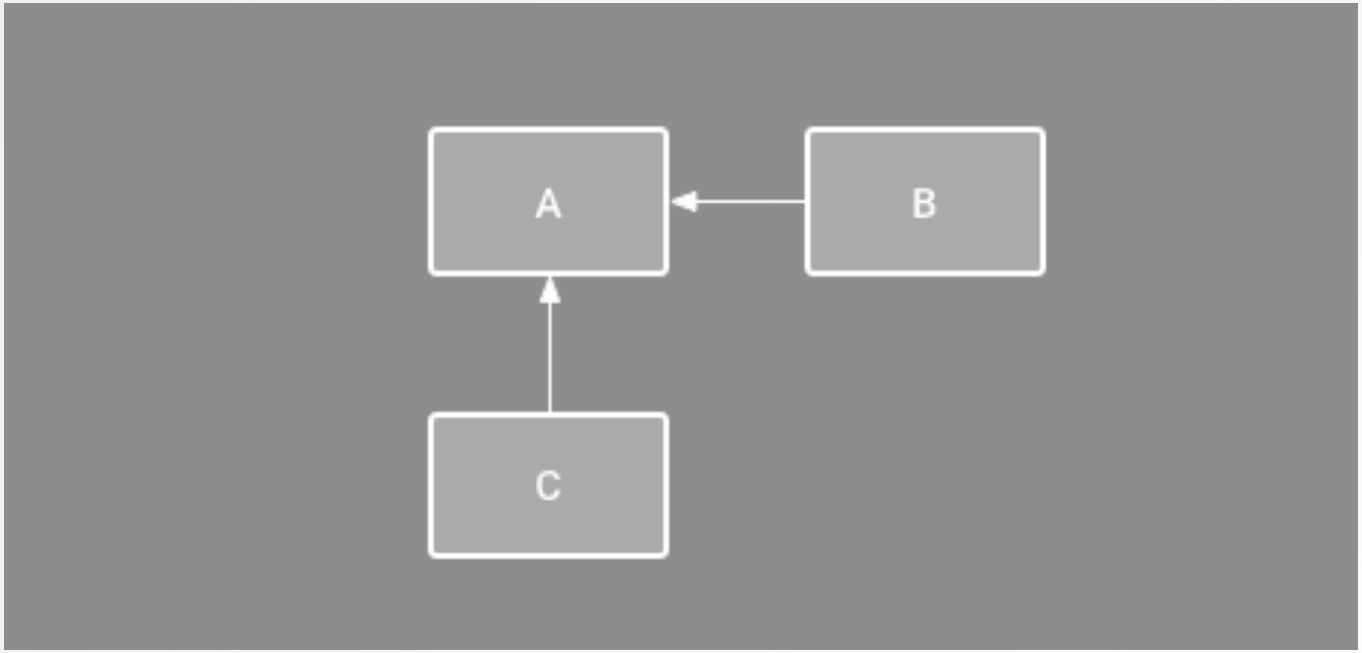
- Constrain the side of a view to the corresponding edge of the layout.
  - The ***left side of the view*** is connected to the ***left edge of the parent*** layout.
  - We can define the **distance** from the edge with margin.





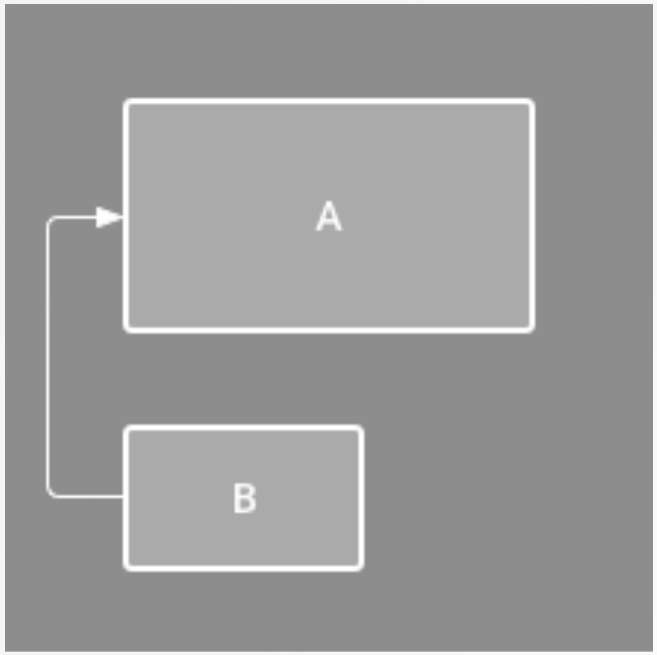
Order position

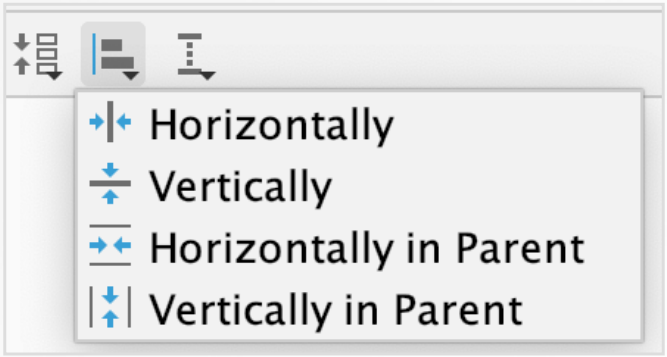
- Define the order of appearance for 2 views, either vertically or horizontally.
  - B is constrained to always be to the right of A, and C is constrained below A.



Alignment

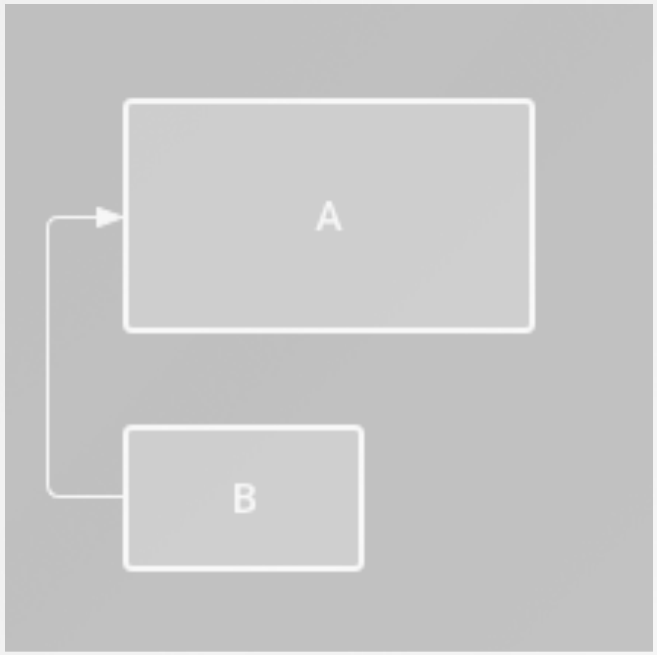
- Align the edge of a view to the same edge of another view.
  - The left side of B is aligned to the left side of A. If you want to align the view centers, create a constraint on both sides.
  - You can also select all the views you want to align, and then click **Align** in the toolbar to select the alignment type.

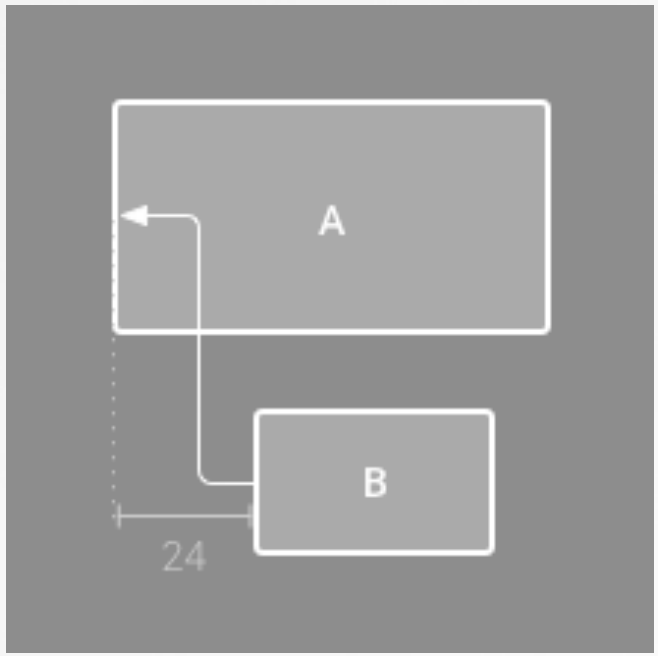




Alignment

- Align the edge of a view to the same edge of another view.
  - The left side of B is aligned to the left side of A.
  - And you can add margin

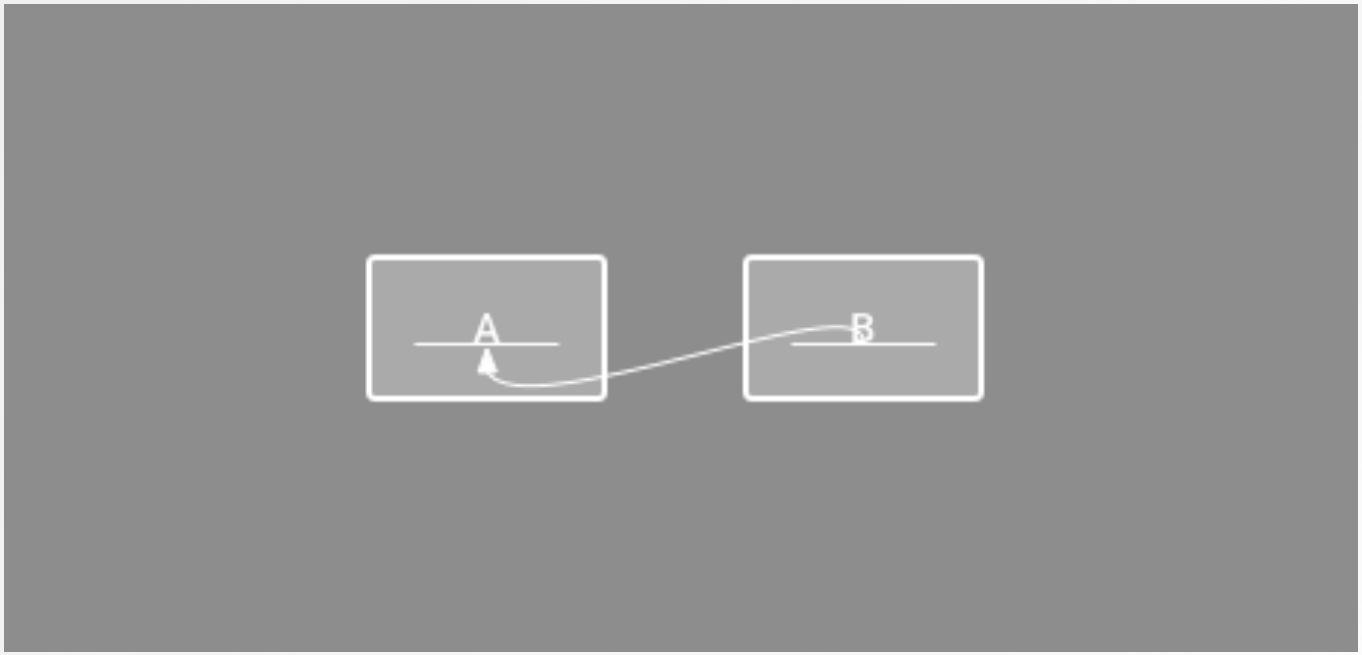




constrained view's margin

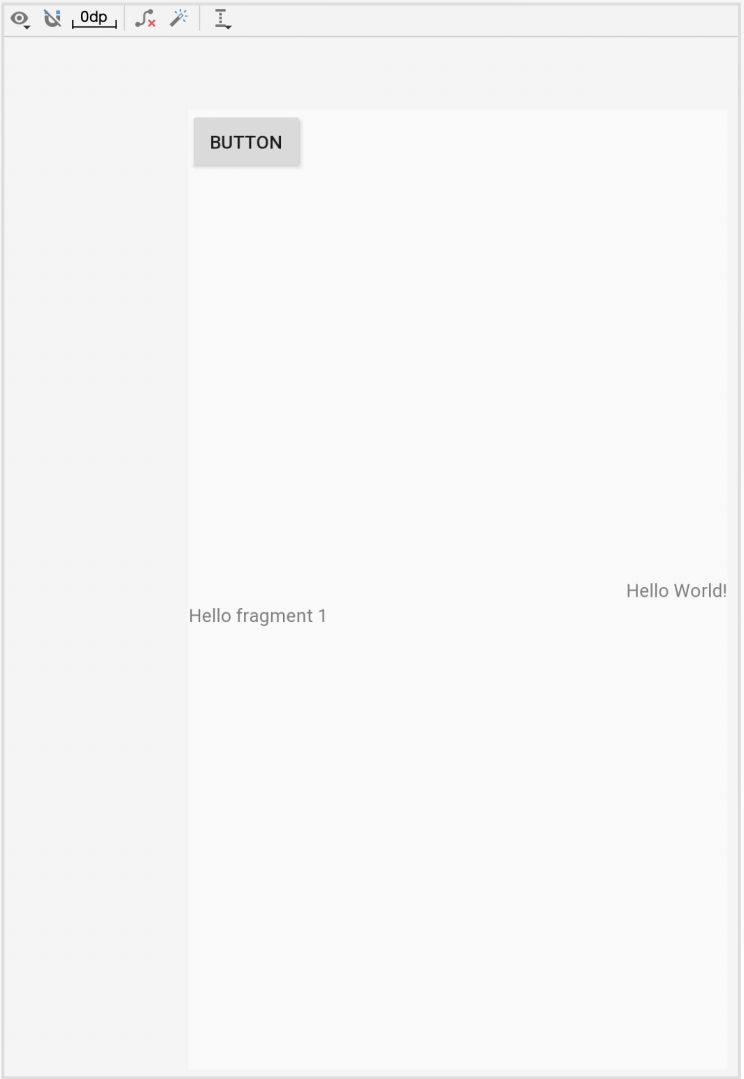
Baseline alignment

- Align the text baseline of a view to the text baseline of another view.
  - The first line of B is aligned with the text in A.



Constrain to a guideline

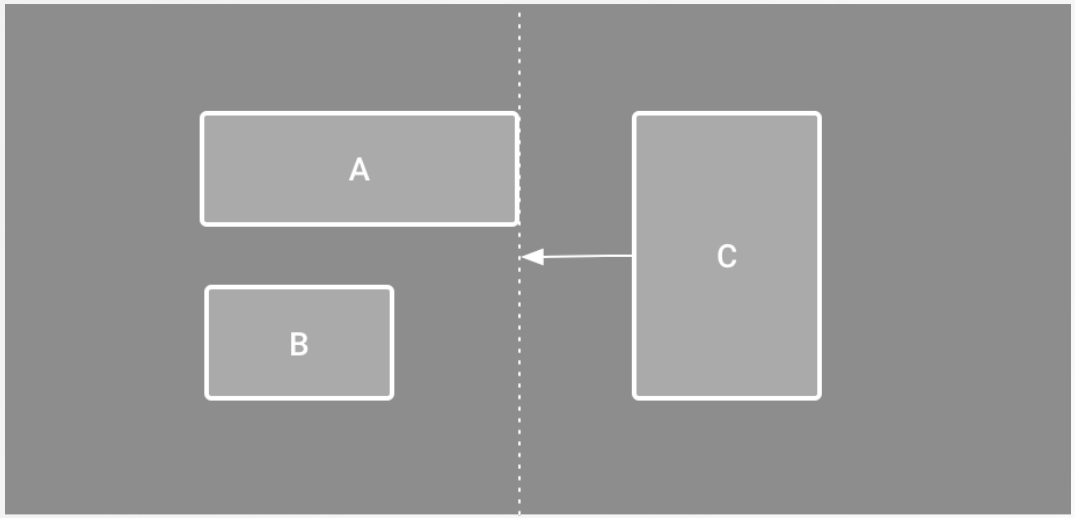
- We can add a ***vertical or horizontal guideline*** to which you can constrain views,
  - The guideline will be invisible to app.
  - We can position the guideline within the layout based on either ***dp units or percent***, relative to the layout's edge.
- Click **Guidelines**  in the toolbar, and then click either **Add Vertical Guideline** or **Add Horizontal Guideline**.
  - Drag the dotted line to reposition it
  - Click the circle at the edge of the guideline to toggle the measurement mode.

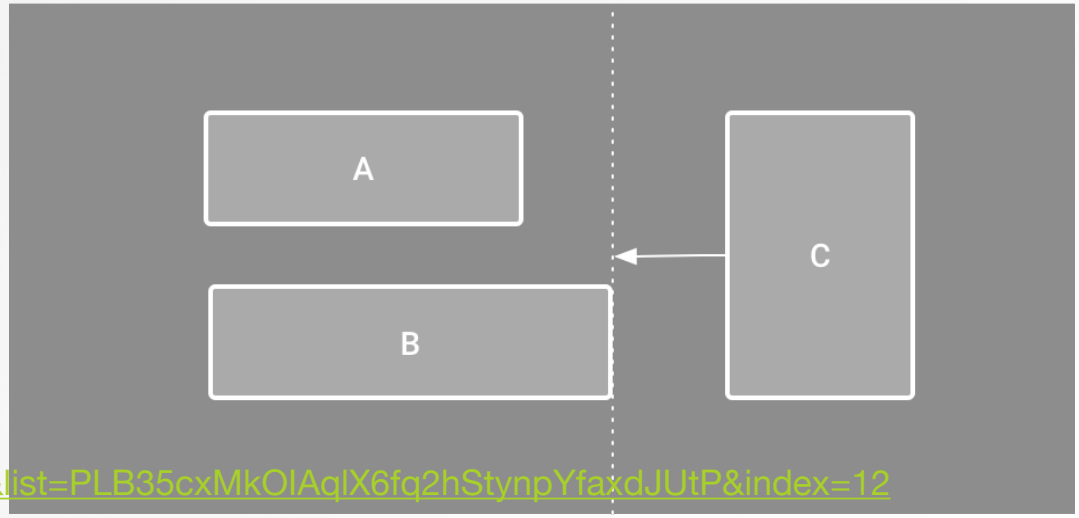


Constrain to a barrier

- Similar to a guideline, a barrier is an invisible line that you can constrain views to.
- A barrier does not define its own position; instead, the barrier position moves based on the position of views contained within it.

  (ConstraintLayout - BARRIERS.mp4)

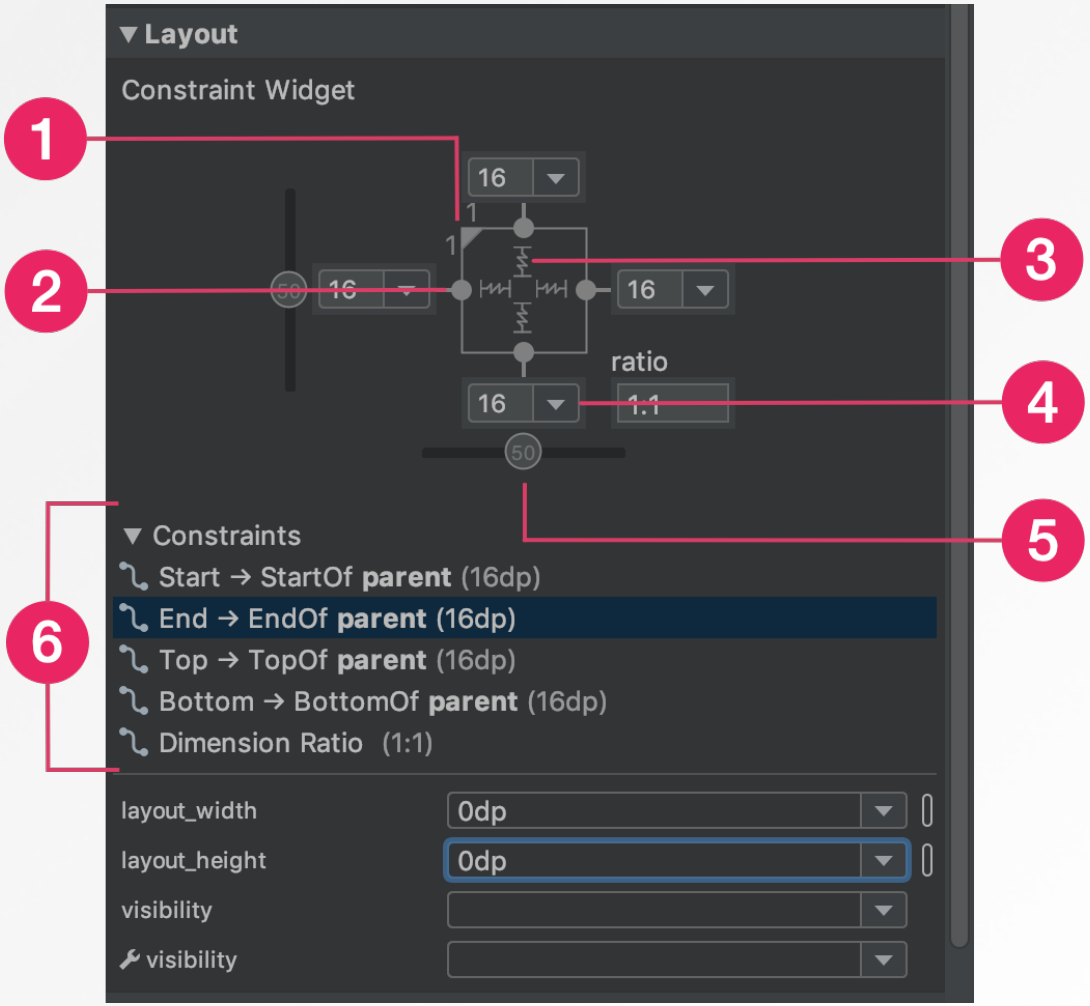




https://www.youtube.com/watch?v=bELy_UGEoVo&list=PLB35cxMkOIAqlX6fq2hStynpYfaxdJUtP&index=12

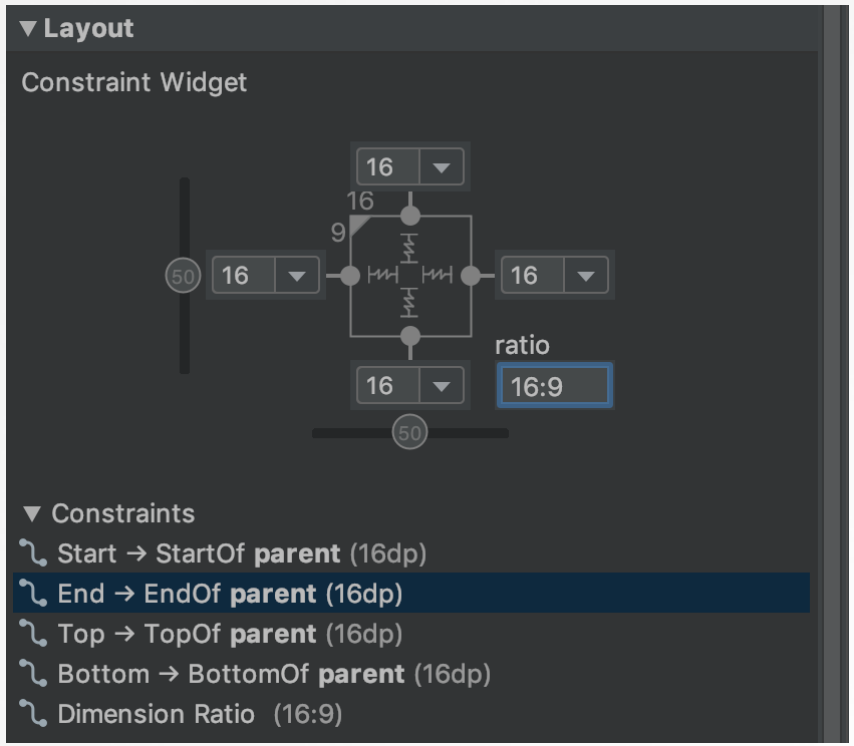
Adjust the view size

- When selecting a view, the **Attributes** window includes controls for:
  - 1 size ratio,
  - 2 deleting constraints,
  - 3 height/width mode,
  - 4 margins,
  - 5 constraint bias,
  - 6 constraint list.



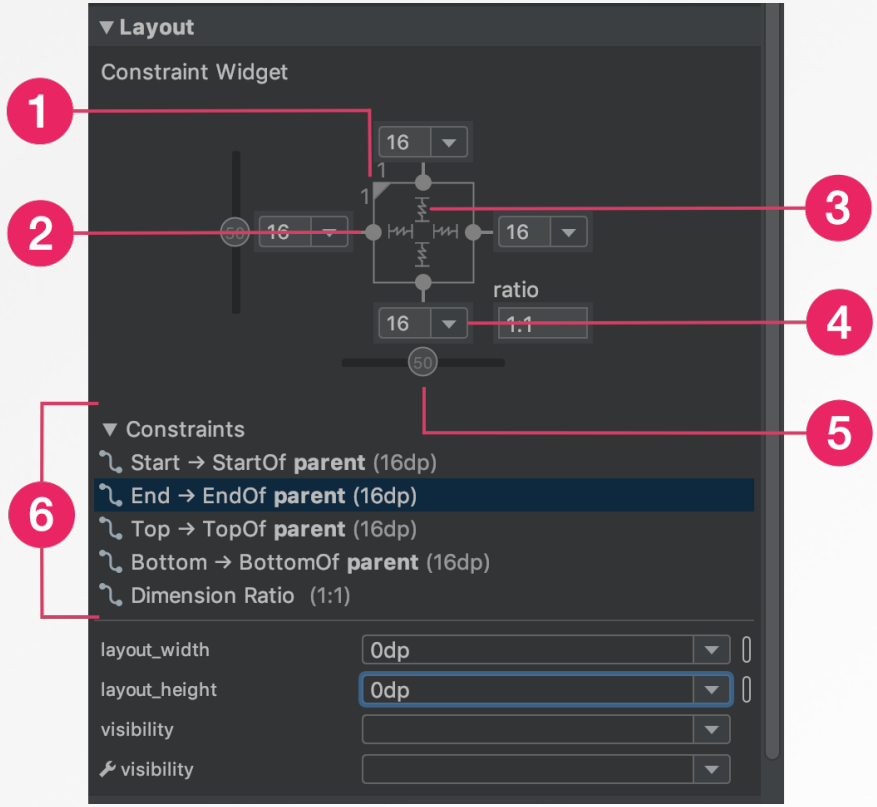
Adjust the view size (cont.)

- 1 size ratio:
  - You can set the view size to a ratio such as 16:9 if at least one of the view dimensions is set to "match constraints" (0dp).
  - To enable the ratio, click **Toggle Aspect Ratio Constraint** and then enter the ***width***:***height*** ratio in the input that appears.

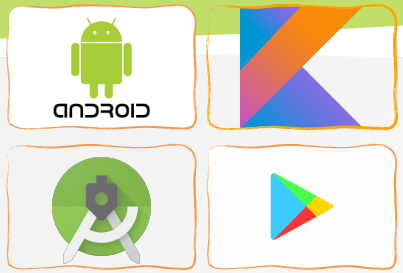


Adjust the view size (cont.)

- 3 height/width mode,
  - **Fixed**: You specify a specific dimension in the text box below or by resizing the view in the editor.
  - **Wrap Content**: The view expands only as much as needed to fit its contents.
  - **Match Constraints**: The view expands as much as possible to meet the constraints on each side (after accounting for the view's margins).

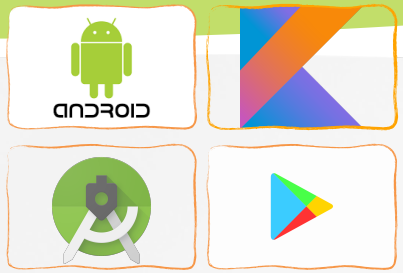


Automatically create constraints

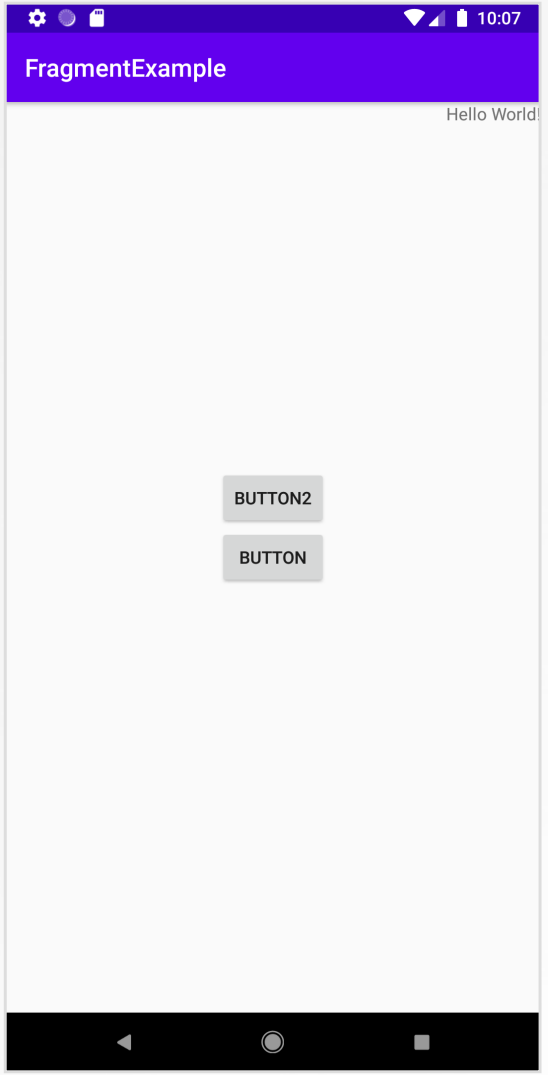


- We can move each view into the positions you desire, and then click **Infer Constraints**  to automatically create constraints.
  - **Infer Constraints** scans the layout to determine the most effective set of constraints for all views. It makes a best effort to constrain the views to their current positions while allowing flexibility.
- **Autoconnect to parent**  automatically creates 2 or more constraints for each view as you add them to the layout, but only when it's appropriate to constrain the view to the parent layout. ***Autoconnect*** does not create constraints to other views in the layout.
  - Autoconnect is disabled by default. 

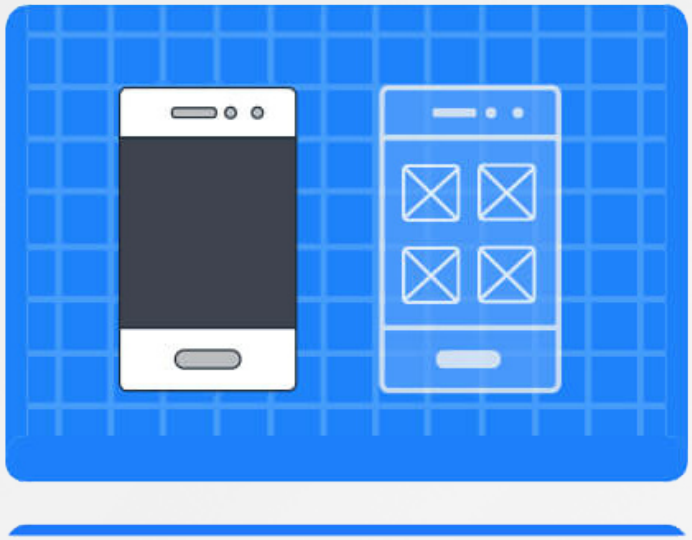
Constraint Layout Programmatically



```
val constraintLayout = findViewById<ConstraintLayout>(R.id.constraintLayout)

val button = Button(this)
button.text = "Button"
button.id = ID_1

val layoutParams = ConstraintLayout.LayoutParams(ConstraintLayout.LayoutParams.WRAP_CONTENT,
    ConstraintLayout.LayoutParams.WRAP_CONTENT)
button.layoutParams = layoutParams
constraintLayout.addView(button)

val button2 = Button(this)
button2.text = "Button2"
button2.id = ID_2
constraintLayout.addView(button2)

val constraintSet = ConstraintSet()
//Copy all the previous constraints present in the constraint layout.
constraintSet.clone(constraintLayout)

constraintSet.connect(ID_1, ConstraintSet.TOP, ConstraintSet.PARENT_ID, ConstraintSet.TOP)
constraintSet.connect(ID_1, ConstraintSet.BOTTOM, ConstraintSet.PARENT_ID, ConstraintSet.BOTTOM)
constraintSet.connect(ID_1, ConstraintSet.LEFT, ConstraintSet.PARENT_ID, ConstraintSet.LEFT)
constraintSet.connect(ID_1, ConstraintSet.RIGHT, ConstraintSet.PARENT_ID, ConstraintSet.RIGHT)

constraintSet.connect(ID_2, ConstraintSet.BOTTOM, ID_1, ConstraintSet.TOP)
constraintSet.connect(ID_2, ConstraintSet.LEFT, ConstraintSet.PARENT_ID, ConstraintSet.LEFT)
constraintSet.connect(ID_2, ConstraintSet.RIGHT, ConstraintSet.PARENT_ID, ConstraintSet.RIGHT)
constraintSet.applyTo(constraintLayout)
```



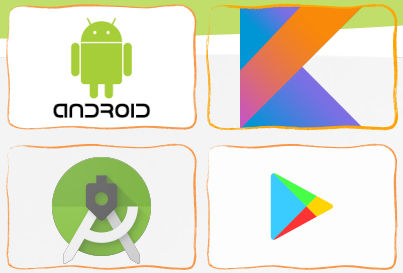
Demo



Kotlin

Q&A

Reference



- https://kotlinlang.org/docs/reference/
- Peter Späth, Learn Kotlin for Android Development, 2019
- Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- Peter Späth, Pro Android with Kotlin, 2019
- Milos Vasic, Mastering Android Development with Kotlin, 2017
- Victor Matos, Mobile Application Development course, Cleveland State University
- https://developer.android.com/guide/